



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A Project Report
On
Scalable Multi-hop LoRa Networks for Intelligent Forest Monitoring**

Submitted By:

Chitra Raj Joshi (THA080BEI018)
Keshar Singh Sunar (THA080BEI022)
Prabesh Parajulee (THA080BEI029)
Santosh Gadtaula (THA080BEI042)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

August, 2026



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

Minor Project Mid-Term Report

On

Scalable Multi-hop LoRa Networks for Intelligent Forest Monitoring

Submitted By:

Chitra Raj Joshi (THA080BEI018)

Keshar Singh Sunar (THA080BEI022)

Prabesh Parajulee (THA080BEI029)

Santosh Gadtaula (THA080BEI042)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In partial fulfillment for the award of the
Bachelor's Degree in Electronics and Communication Engineering

Under the Supervision of

Er. Saroj Shakya

August, 2026

DECLARATION

We hereby declare that the report of the project entitled “Scalable Multi-hop LoRa Networks for Intelligent Forest Monitoring” which is being submitted to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus, in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in Electronics and Communication Engineering, is a bona fide report of the work carried out by us. The materials contained in this report have not been submitted to any University or Institution for the award of any degree and we are the only authors of this complete work and no sources other than the listed ones have been used in this work.

Chitra Raj Joshi (THA080BEI018) _____

Keshar Singh Sunar (THA080BEI022) _____

Prabesh Parajulee (THA080BEI029) _____

Santosh Gadtaula (THA080BEI042) _____

Date: August, 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus, a minor project work entitled “Scalable Multi-hop LoRa Networks for Intelligent Forest Monitoring” submitted by Chitra Raj Joshi, Keshar Singh Sunar, Prabesh Parajulee and Santosh Gadtaula in partial fulfillment for the award of Bachelor’s Degree in Electronics and Communication Engineering. The Project was carried out under special supervision and within the time frame prescribed by the syllabus.

We found the students to be hardworking, skilled and ready to undertake any related work to their field of study and hence we recommend the award of partial fulfillment of Bachelor’s degree of Electronics and Communication Engineering.

Project Supervisor

Er. Saroj Shakya

Department of Electronics and Computer Engineering, Thapathali Campus

External Examiner

Examiner’s Name

Department of Electronics and Computer Engineering

Project Co-ordinator

Project Co-ordinator’s Name

Department of Electronics and Computer Engineering, Thapathali Campus

Head of the Department

Er. Umesh Kant Ghimire

Department of Electronics and Computer Engineering, Thapathali Campus

August, 2026

COPYRIGHT

The authors have agreed that the library, Department of Electronics and Computer Engineering, Thapathali Campus, may make this report freely available for inspection. Moreover, the authors have agreed that the permission for extensive copying of this project work for scholarly purpose may be granted by the professor/lecturer, who supervised the project work recorded herein or, in their absence, by the head of the department. It is understood that the recognition will be given to the authors of this report and to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus in any use of the material of this report. Copying of publication or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, IOE, Thapathali Campus and authors' written permission is prohibited.

Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to Department of Electronics and Computer Engineering, IOE, Thapathali Campus.

ACKNOWLEDGEMENT

This project would not have taken shape without the support of several people and institutions, and we would like to acknowledge them here.

First, we are thankful to the **Department of Electronics and Computer Engineering at Thapathali Campus**, Institute of Engineering, Tribhuvan University, for making this minor project a part of our academic curriculum. We owe a special thanks to our supervisor, **Er. Saroj Shakya**, for his time and patience throughout the planning of this project. His feedback pushed us to think more carefully about the practical challenges of deploying acoustic sensing in real forest environments, and his suggestions helped us narrow down a fairly broad idea into something we could realistically build and test.

We are also thankful to our classmates and friends, with whom countless informal discussions helped us work through design choices, troubleshoot ideas, and stay motivated through the early planning stages of this project.

Chitra Raj Joshi (THA080BEI018)

Keshar Singh Sunar (THA080BEI022)

Prabesh Parajulee (THA080BEI029)

Santosh Gadtaula (THA080BEI042)

ABSTRACT

Forests face increasing threats from wildfires, illegal logging, and poaching, which lead to the demand for reliable, energy-efficient monitoring systems capable of operating in remote environments. This project proposes a multi-hop LoRa-based wireless sensor network for intelligent forest monitoring that integrates environmental sensing, edge-based machine learning, and low-power communication. The proposed system consists of multiple sensor nodes and a gateway controlled by Esp32 microcontroller. The sensor node consists of multiple sensor like temperature, humidity, and gas (CO₂) sensors, which combine multisensory fusion to give the fire risk score. The acoustic data captured by a MEMS microphone is processed through a TinyML inference model deployed directly on the node, enabling on-device classification of illegal activity sounds such as chainsaws and gunshots. A network of LoRa-enabled sensor nodes collaboratively communicates to a central gateway, which aggregates real-time data and forwards critical alerts to a live web dashboard for monitoring and response. Nodes operate in deep-sleep mode and wake periodically, and Inter-node communication is handled through the LDSE protocol, which enables multi-hop networking and extends coverage beyond a single gateway range, delivering a low-cost, scalable, and energy-efficient solution for forest monitoring in community forests and protected areas. The proposed system is expected to improve acoustic classification, improve fire detection accuracy, low power and energy efficient device for remote monitoring.

Keywords: Wireless Sensor Networks (WSN), Multi-hop LoRa Communication, Edge AI, TinyML, Energy-Efficient Routing, LDSE Protocol, Forest Monitoring, Gateway, Multisensor fusion.

TABLE OF CONTENTS

DECLARATION	i
CERTIFICATE OF APPROVAL	ii
COPYRIGHT	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
List of Figures	viii
List of Tables	ix
List of Abbreviations	x
1 INTRODUCTION	1
1.1 Background	1
1.2 Motivation	2
1.3 Problem Definition	3
1.4 Objectives	3
1.5 Report Organization	4
1.6 Scope and Applications	5
1.6.1 Scope	5
1.6.2 Applications	6
2 LITERATURE REVIEW	7
3 REQUIREMENT ANALYSIS	11
3.1 Hardware Requirements	11
3.2 Software Requirements	11
3.3 Feasibility Study	13
3.3.1 Technical Feasibility:	13
3.3.2 Economic Feasibility:	13
3.3.3 Operational Feasibility:	13
3.3.4 Time Feasibility:	14
4 SYSTEM ARCHITECTURE AND METHODOLOGY	15
4.1 System Overview and Architecture	15
4.2 Communication System Design	15
4.2.1 LoRa Communication Fundamentals	16
4.2.2 LoRa Communication Parameters	16
4.2.3 Conventional LoRaWAN Architecture	17
4.2.4 Limitations of Conventional LoRaWAN	18
4.2.5 LDSE Protocol Overview	18
4.2.6 Architecture of LDSE	18
4.2.7 Time Synchronization and Epoch Structure	19

4.2.8	Parent Selection and Routing Mechanism	20
4.2.9	Multi-Channel Communication	20
4.2.10	Power Consumption Model	21
4.2.11	Forest Radio Propagation Considerations	21
4.2.12	Protocol-Centric Implementation	22
4.2.13	Operational Challenges and Mitigation Strategies	22
4.2.14	Acoustic Dataset: FSC22	23
4.2.15	Acoustic Feature Extraction	24
4.2.16	Memory-Constrained Model Architecture Design	25
4.2.17	CNN Architecture	26
4.2.18	Quantization Strategy	27
4.2.19	On-Device Deployment Pipeline	28
4.2.20	Fire Risk Scoring	30
4.3	System Data Flow	31
4.4	Overall Working Principle	32
5	IMPLEMENTATION DETAILS	33
5.1	Acoustic Threat Detection Implementation	33
5.1.1	Training Pipeline	33
5.1.2	INT8 Quantization	34
5.2	LoRa Network Simulation Implementation	36
5.2.1	Basic LoRa Node Communication Implementation	37
6	RESULT AND ANALYSIS	39
6.1	Environmental Sensor-Based Fire Risk Assessment	39
6.2	LoRa Communication Simulation Results	39
6.3	Current Limitations	40
6.4	Current Project Progress	41
6.5	Acoustic Threat Detection Results	41
7	FUTURE ENHANCEMENTS	44
8	CONCLUSION	48
9	APPENDICES	49
	Appendix A: PROJECT SCHEDULE	49
	Appendix B: PROJECT BUDGET	50
	Appendix C: HARDWARE PROTOTYPE SETUP	51
	REFERENCES	53

LIST OF FIGURES

Figure 4-1: Proposed System Architecture	15
Figure 4-2: LDSE protocol network hierarchy	19
Figure 4-3: LDSE Protocol Epoch Structure	20
Figure 4-4: Sequential Synchronization and Operational Flowchart	22
Figure 4-5: Block diagram of the deployed Tiny CNN, showing the three convolutional blocks and the classification head	27
Figure 4-6: On-device acoustic inference pipeline	29
Figure 4-7: Proposed System Dataflow	31
Figure 5-1: Log-Mel spectrograms for representative classes.	34
Figure 5-2: LoRaWAN communication using OMNeT++	37
Figure 5-3: LoRa communication between two LoRa end devices in OMNeT++	38
Figure 6-1: Per-class precision, recall, and F1-score for the deployed tiny model	42
Figure 6-2: Confusion matrix for the deployed tiny INT8 model)	43
Figure 9-1: Schematic diagram of the sensing layer.. . . .	51

LIST OF TABLES

Table 4-1:	FSC22 sub-class to project-class mapping	24
Table 4-2:	Model architecture design comparison	25
Table 4-3:	Layer-by-layer summary of the deployed Tiny CNN	27
Table 5-1:	Deployed model size and parameter count by architecture variant. . .	35
Table 5-2:	ESP32-S3 runtime memory budget	35
Table 6-1:	Environmental Sensor Testing Results	39
Table 6-2:	Current Project Status	41
Table 6-3:	Classification report – Tiny INT8 model on FSC22 test split.	42
Table 9-1:	Project Schedule Gantt Chart	49
Table 9-2:	Project Budget	50

List of Abbreviations

AI	Artificial Intelligence
CNN	Convolutional Neural Network
DHT22	Digital Humidity and Temperature Sensor (Model 22)
DSP	Digital Signal Processing
ESP32-S3	Espressif Systems 32-bit Microcontroller (Series 3)
FSC22	Forest Sound Classification Dataset 2022
FTSP	Flooding Time Synchronization Protocol
int8	8-bit Integer Quantization
IoT	Internet of Things
IRE	Implicit Route Exploration
JSON	JavaScript Object Notation
LDSE	Layered Dynamic Synchronization Energy-saving
LoRa	Long Range (radio modulation technology)
LoRaWAN	Long Range Wide Area Network
MEMS	Micro-Electro-Mechanical Systems
MFCC	Mel-Frequency Cepstral Coefficient
MHz	Megahertz
ML	Machine Learning
MQ135	Air Quality Sensor Module (Model 135)
RF	Random Forest
SX1278	Semtech LoRa Transceiver (Model 1278)
TFLite	TensorFlow Lite
TinyML	Tiny Machine Learning
WSN	Wireless Sensor Network

1 INTRODUCTION

1.1 Background

Forests are among the most critical ecosystems on Earth, providing ecological stability, biodiversity, and carbon sequestration services that are vital to sustaining life. However, they face growing and compounding threats from illegal logging, poaching, and wildfires, particularly in densely forested regions such as Nepal’s community forests and protected areas. Conventional patrol-based monitoring cannot cover large remote areas continuously, and incidents often go undetected until significant environmental and economic damage has already occurred. Nepal experiences approximately 40,000 hectares of burned forest area annually [1][2], with more than 78% to 86 % of fire incidents occurring between March and May during the dry season[3]. Historical analyses also indicate that the total burned area has increased by approximately 0.6% per year since 2001, highlighting the growing need for early detection and continuous monitoring systems. Human activities combined with prolonged dry conditions remain the dominant contributors to forest fire occurrence.

Low-power wide-area networks such as LoRa support long-range, energy-efficient wireless links that are well suited for deployment in dense and geographically challenging forest environments. Prior systems have used sound and motion sensing for illegal logging and wildlife protection, but many rely on single-hop star topologies that limit coverage range and require dense gateway infrastructure. Machine learning deployed at the edge further enables intelligent, real-time decision-making without dependence on cloud connectivity, which is often unavailable in remote forests.

Acoustic sensing has emerged as a particularly effective way to detect illegal logging and poaching activity, since sounds such as chainsaw operation, axe strikes, and gunshots are highly distinctive and can be identified even in dense forest canopy where visual monitoring is impractical. Early acoustic detectors relied on simple threshold-based or hand-crafted feature methods, which proved unreliable amid the variability of natural forest noise. Convolutional neural networks trained on Mel-spectrogram rep-

representations have since substantially improved classification accuracy, but such models have traditionally required cloud servers or power-hungry edge devices to run, making them unsuitable for standalone, battery-powered deployment in remote forest environments. Recent advances in TinyML, enabling quantized neural networks to run directly on low-power microcontrollers, make it possible to perform this acoustic classification entirely on-device, removing the dependence on continuous connectivity while keeping node power consumption low enough for long-term field deployment.

This project addresses the coverage and intelligence gap by combining multi-hop LDSE networking with edge artificial intelligence on the ESP32-S3 microcontroller for unified fire detection, acoustic threat identification, and network scalability in a single low-cost platform.

1.2 Motivation

Forests play a critical role in maintaining ecological balance, yet they remain vulnerable to threats such as wildfires, illegal logging, and poaching. These activities often occur in remote areas where continuous human supervision is impractical, causing incidents to go undetected until significant environmental and economic damage has already occurred. Existing monitoring approaches are often limited by inadequate coverage, delayed detection, and the challenges of operating in large, inaccessible forest regions. Therefore, there is a growing need for an intelligent and scalable monitoring system capable of providing early threat detection and reliable coverage across vast forest environments.

Nepal's community forests and protected reserves, including those in the Terai and mid-hill regions, face persistent threats from unauthorized timber extraction, wildlife poaching, and seasonal wildfires. The absence of affordable and autonomous monitoring infrastructure means that forest rangers must physically patrol vast areas, a process that is both costly and ineffective. An integrated IoT system combining LoRa communication and on-device machine learning can transform how these threats are detected and reported, enabling faster response times, improving conservation outcomes, and

contributing to the long-term protection of forest ecosystems.

1.3 Problem Definition

Most existing LoRaWAN-based forest monitoring systems rely on a single-hop communication model where sensor nodes must directly reach a central gateway, which significantly limits coverage in dense and geographically challenging forest environments. These systems often depend on continuous or frequent data transmission, leading to high energy consumption and reduced node lifetime, particularly in remote deployments where maintenance is difficult. In addition, many traditional approaches use simple threshold-based detection mechanisms that are highly sensitive to environmental noise, resulting in frequent false alarms or missed detections when thresholds are not properly tuned.

Furthermore, existing designs are typically optimized for a single type of event such as fire detection, lacking the capability to simultaneously monitor multiple threats like illegal logging or human intrusion. Sound-based detection systems in the literature often perform inference on cloud servers, making them unsuitable for forest environments with limited or no connectivity. The absence of efficient in-network processing and reliance on raw data streaming to central servers further increases communication overhead and limits scalability in large-scale forest monitoring scenarios.

1.4 Objectives

The project objectives are:

- To develop an intelligent forest monitoring node that computes a real-time fire risk score from fused environmental sensor data and detects acoustic threats by deploying a quantized TinyML model on the edge to identify illegal sounds such as chainsaws, axes, and gunshots.
- To deploy the LDSE multi-hop LoRa protocol and benchmark its coverage extension and energy efficiency improvements against conventional single-hop Lo-

RaWAN.

1.5 Report Organization

This report is organized as follows:

- **Chapter 1: Introduction** presents the background, motivation, problem definition, objectives, project scope and applications, and the organization of this report.
- **Chapter 2: Literature Review** summarizes related work on forest fire detection, acoustic threat detection, and multi-hop LoRa communication.
- **Chapter 3: Requirement Analysis** presents the hardware and software requirements of the system along with a feasibility study.
- **Chapter 4: System Architecture and Methodology** describes the overall system architecture, block diagram, and the methodology adopted for the design.
- **Chapter 5: Implementation Details** explains the details of the hardware and software implementation of the prototype.
- **Chapter 6: Result and Analysis** presents and discusses the results obtained from the testing and simulation.
- **Chapter 7: Future Enhancements** outlines the tasks that remain to complete the project.
- **Chapter 8: Conclusion** summarizes the findings of the project.
- **Appendices** contain the project schedule, project budget, and hardware prototype setup details.

1.6 Scope and Applications

1.6.1 Scope

The proposed prototype consists of two forest sensor nodes and one gateway. The sensor nodes are built on the ESP32-S3 platform and communicate using LoRa technology, while the LDSE multi-hop routing protocol enables reliable data transmission over distances exceeding 2 km when a direct gateway connection is unavailable. The gateway receives sensor data and transmits it to a web-based dashboard through Wi-Fi, allowing real-time monitoring and alert visualization. System evaluation includes laboratory testing to validate sensor readings, assess acoustic model inference accuracy, and verify network communication. In addition, limited outdoor field trials will be conducted to evaluate communication reliability, overall system performance, and energy efficiency. The prototype is limited to a single-gateway architecture, and large-scale multi-gateway deployments and cellular backhaul integration are beyond the scope of this work.

1.6.2 Applications

The proposed system is designed for deployment in community forests and protected areas in Nepal, where continuous environmental monitoring is required but conventional monitoring infrastructure is often unavailable or too costly. Its low-cost, low-power, and autonomous design makes it suitable for long-term operation in remote forest environments. The system can support early detection of forest fires, illegal logging activities, and other environmental events through real-time sensor monitoring and acoustic analysis. Additionally, the web-based dashboard enables forest authorities and conservation organizations to remotely monitor forest conditions and respond promptly to detected incidents, thereby improving forest management and environmental protection.

2 LITERATURE REVIEW

The concept of distributed environmental monitoring using wireless sensor networks (WSNs) emerged in the late 1990s with pioneering projects such as Smart Dust and the Berkeley Motes. Early systems demonstrated the feasibility of deploying low-power sensor nodes for habitat monitoring, soil moisture measurement, and fire weather forecasting. However, these first-generation WSNs were constrained by limited communication range (typically tens to a few hundred meters), high per-node cost, and reliance on single-hop transmission to a central sink. As a result, covering large forest areas required dense node placement or expensive repeater infrastructure, which was impractical for remote community forests in developing regions[4]. This historical limitation motivates the need for a long-range, low-power wireless technology that can extend coverage without proportional infrastructure cost.

Energy consumption quickly became the defining constraint for long-term environmental monitoring. In the early 2000s, researchers introduced duty-cycling and low-power MAC protocols (e.g., S-MAC, T-MAC) that allowed nodes to remain in deep sleep for most of their operational lifetime, waking periodically to sense and transmit. Later advances integrated solar harvesting and ultra-low-power microcontrollers, pushing node lifetimes from weeks to months. Despite these improvements, many systems still relied on frequent transmission of raw sensor data, and synchronization overhead in duty-cycled networks remained a challenge. This project addresses this gap by implementing deep-sleep scheduling (targeting $\sim 7\mu\text{A}$ idle current) coupled with event-driven wake-up from acoustic interrupts, reducing unnecessary transmissions and extending battery life beyond conventional periodic sampling approaches.[5]

To overcome the limited range of single-hop WSNs, researchers adapted multi-hop ad-hoc networking principles from MANETs to sensor networks. Protocols such as LEACH (Low-Energy Adaptive Clustering Hierarchy) and later AODV (Ad-hoc On-Demand Distance Vector) [6] enabled nodes to relay data through intermediate neighbors. However, these reactive protocols introduced significant control overhead (route discovery floods, periodic beacons) that consumed energy and reduced scalability. Clus-

tering approaches improved energy efficiency but often required global topology knowledge and frequent re-clustering. The central challenge remained: how to achieve reliable multi-hop coverage with minimal routing overhead and low synchronization cost. This project directly confronts this gap by adopting the LDSE (Layered Dynamic Synchronization Energy-saving) protocol, which limits route discovery overhead to approximately 5% of total traffic and coordinates synchronized sleep-relay cycles, thereby extending coverage beyond 2 km without the broadcast storms typical of AODV.

The introduction of the LoRa physical layer by Semtech in 2012 [7] and the subsequent LoRaWAN standard (2015) [8] marked a paradigm shift for remote environmental monitoring. LoRa provides ultra-long-range communication (several kilometers in line-of-sight, 1–2 km in dense forest) at extremely low power, making it ideal for forest IoT applications. However, the standard LoRaWAN architecture is a star-of-stars topology [9], in which each end node communicates directly with a gateway. While simple and energy-efficient, this topology forces a trade-off: covering large forest areas requires many gateways, driving up cost and deployment complexity. Multi-hop extensions to LoRa have been proposed in academic literature, but few have been validated in real forest deployments with energy-constrained nodes. This project fills this gap by implementing a multi-hop LoRa network using the LDSE protocol, which enables low-cost coverage extension without dense gateway infrastructure a critical requirement for community forests in Nepal.

Detecting illegal logging (chainsaws, axes) and poaching (gunshots) via sound has been explored since the early 2010s. Initial systems used simple thresholding or support vector machines (SVMs) on hand-crafted features (zero-crossing rate, spectral centroid), achieving moderate accuracy but suffering from high false alarms due to natural forest noise. The introduction of Mel-spectrogram features combined with convolutional neural networks (CNNs) significantly improved classification performance. Notably, Kahl et al. (2023) introduced the FSC22 dataset, a benchmark for forest sound classification, and demonstrated that CNNs [10] on Mel-spectrograms achieve state-of-the-art results. Yet, nearly all such systems perform inference on cloud servers or powerful edge devices (Raspberry Pi), requiring continuous connectivity or high power. This project addresses this by deploying a quantized TinyML CNN directly on the ESP32-S3 mi-

crocontroller, enabling on-device classification of axes, chainsaws, gunshots, and hand-saws (with a background class for non-threat forest sounds) without cloud dependency, while staying within 512 KB SRAM and consuming minimal energy.

Early wildfire detection systems relied on single-threshold triggers from temperature or smoke sensors, which often generated false alarms during normal daily fluctuations. To improve robustness, researchers proposed multi-sensor fusion, combining temperature, humidity, and gas (CO, CO₂, smoke) measurements [11]. Methods ranged from simple weighted scoring to fuzzy logic and machine learning (random forests, neural networks) [12]. While effective, most fusion models were designed for single-event detection (fire only) and did not integrate with other threat detection (logging, poaching). Moreover, these systems frequently transmitted raw sensor data continuously, increasing energy consumption. This project bridges this gap by computing a composite fire-risk score using a Random Forest–inspired weighted deviation formula (with coefficients derived from sensor correlation analysis) and transmitting alerts only when the risk exceeds a threshold, thereby reducing communication overhead while maintaining detection accuracy.

The emergence of TinyML running machine learning models on resource-constrained microcontrollers has been enabled by frameworks like TensorFlow Lite Micro and Edge Impulse. These tools support model quantization (e.g., int8), reducing model size and inference latency by $\sim 75\%$ compared to float32, making CNNs feasible on platforms with limited SRAM (e.g., ESP32-S3’s 512 KB). Prior work has demonstrated on-device keyword spotting and wildlife audio classification, but few studies have deployed TinyML for simultaneous acoustic threat detection (logging/gunshots) and environmental sensor fusion in a single, low-power node *cite ayankoso2024development*. This project fills this integration gap by independently training, quantizing, and deploying a 5-class acoustic CNN using TensorFlow/Keras and TensorFlow Lite alongside a fire-risk scoring module on the same ESP32-S3, achieving sub 200 ms inference latency and enabling true edge intelligence without cloud connectivity.

Research Gap

While individual components, such as multi-hop LoRa, acoustic CNNs, fire sensor fusion, and TinyML, have been studied separately, very few systems combine them in a unified, low-power, scalable forest monitoring solution. Existing integrated platforms often rely on single-hop LoRaWAN (limiting coverage) or perform acoustic inference off-node (requiring cloud or gateway processing). To the best of our knowledge, no prior work has embedded the LDSE multi-hop protocol, a quantized acoustic TinyML classifier, and a sensor-fusion fire-risk model on a single ESP32-S3 node specifically for community forest monitoring in developing countries like Nepal. This project directly targets this research gap by delivering a prototype that integrates all three capabilities LDSE-based multi-hop LoRa, on-device acoustic threat detection, and real-time fire-risk scoring in a low-cost, solar-assisted platform, validated for energy efficiency and coverage extension.

3 REQUIREMENT ANALYSIS

3.1 Hardware Requirements

The hardware components must support real-time environmental monitoring, local edge-based acoustic processing, and low-power long-range communication. The specific hardware requirements include:

- **Microcontroller (ESP32-S3):** Must feature a dual-core Xtensa LX7 processor running up to 240 MHz with vector instructions (to accelerate TinyML neural network calculations). It requires at least 512 KB of on-chip SRAM and support for external PSRAM to handle audio buffering and spectrogram conversion.
- **LoRa Transceiver (SX1278):** Must operate in the 433 MHz ISM band with a high link budget (up to 170 dB) and high sensitivity to ensure reliable, multi-hop data transmission through dense forest canopies and rugged terrain.
- **Acoustic Sensor (MEMS Microphone):** An omnidirectional digital MEMS microphone (such as the INMP441) using an I2S interface to stream high-fidelity, low-noise audio directly to the ESP32-S3 for acoustic analysis.
- **Environmental Sensors:** An integrated array containing a high-accuracy temperature and humidity sensor (e.g., DHT22 or SHT3x) and a gas sensor (e.g., MQ series) to gather ambient environmental data for fire risk estimation.
- **Power Management System:** A low-dropout (LDO) voltage regulator, a Li-ion/LiPo battery charger circuit, and a solar panel interface to sustain continuous, autonomous outdoor operations.

3.2 Software Requirements

The software stack must support local signal processing, machine learning inference, network routing simulation, and data visualization. The software requirements are clas-

sified as follows:

- **Embedded Development Environment:**

- **ESP-IDF / Arduino IDE:** Used for writing, compiling, and flashing the firmware onto the ESP32-S3.
- **TinyML Framework (TensorFlow Lite for Microcontrollers):** Required to load, optimize, and run the quantized acoustic classification model directly on the microcontroller.

- **Digital Signal Processing (DSP) Libraries:** Libraries used to transform raw audio streams into Mel-spectrograms for model inference.

- **Simulation Tools:**

- **OMNeT++:** Used to model the physical wireless network.
- **FLORA Framework:** An OMNeT++ based simulator utilized to simulate the LoRa physical layer, evaluate packet delivery ratios, and test the LDSE multi-hop routing protocol.
- **ns-3 Network Simulator:** Utilized to simulate the LoRa network, evaluate packet delivery ratio, latency, energy consumption, and network scalability, and validate the performance of the proposed LDSE multi-hop routing protocol.

- **Application and Visualization Software:**

- **Web Dashboard Framework:** A web-based application (built using modern web technologies) to display real-time sensor readings, historical trends, acoustic classification events, and node health status.

3.3 Feasibility Study

3.3.1 Technical Feasibility:

The ESP32-S3 microcontroller provides sufficient computational capability for TinyML inference at the edge, with its Xtensa LX7 dual-core processor and a vector instruction set enabling efficient neural network computation within the available 512 KB SRAM. The SX1278 LoRa transceiver provides long-range communication with a link budget exceeding 170 dB, well suited for the dense vegetation and terrain variation found in Nepalese community forests. The Edge Impulse platform provides a mature and well-documented pipeline for training, quantizing, and deploying CNN models to the ESP32-S3 target, and TensorFlow Lite Micro has been deployed successfully on similar hardware in prior academic work. All hardware components selected for this project are commercially available and have documented community support.

3.3.2 Economic Feasibility:

The estimated total prototype budget of approximately NPR 12,400 positions this system as a highly cost-effective alternative to commercial forest monitoring solutions, which typically require expensive satellite or cellular infrastructure.

The use of commodity hardware components and open-source software stacks including Edge Impulse, TensorFlow Lite Micro, and the Arduino framework eliminates licensing costs. The modular and scalable architecture means that additional forest nodes can be added to expand coverage at a marginal per-node cost well below the gateway cost, making the system economically viable for community forest committees operating under budget constraints.

3.3.3 Operational Feasibility:

The system is designed to operate autonomously with deep-sleep scheduling and event-driven wake-up, requiring minimal human intervention after initial deployment and

configuration. Solar-assisted battery operation can extend node lifetime significantly in remote forest environments where regular maintenance visits are impractical. The real-time web dashboard provides an intuitive interface for forest rangers to monitor alerts, review event histories, and identify geographic patterns of illegal activity without requiring technical expertise. Fail-safe design including confidence thresholding for alert transmission reduces false alarm rates, maintaining the trust of operators in the system's outputs.

3.3.4 Time Feasibility:

The proposed two month project schedule is realistic given the availability of the FSC22 dataset, the Edge Impulse platform's rapid model development capability, the team's familiarity with embedded systems and IoT development, and the modular architecture of the system design. Each phase builds incrementally on the previous one, and the independence of the acoustic model training, fire-risk model development, LDSE protocol implementation, and dashboard development workstreams allows parallel progress across team members.

4 SYSTEM ARCHITECTURE AND METHODOLOGY

This chapter explains the overall system architecture, the hardware components used in each node, the multi-hop communication protocol, and the edge-AI detection strategy that together form the proposed forest monitoring system. The chapter also traces the end-to-end data flow and overall working principle of the network.

4.1 System Overview and Architecture

The proposed system follows four layers: Sensing, Edge Processing, Communication, and Gateway/Cloud. Two forest nodes monitor the environment continuously while most hardware components remain in deep sleep to conserve energy. Nodes wake on a scheduled timer or upon detection of a significant acoustic or environmental event. Figure 4-1 shows the layered architecture of the system.

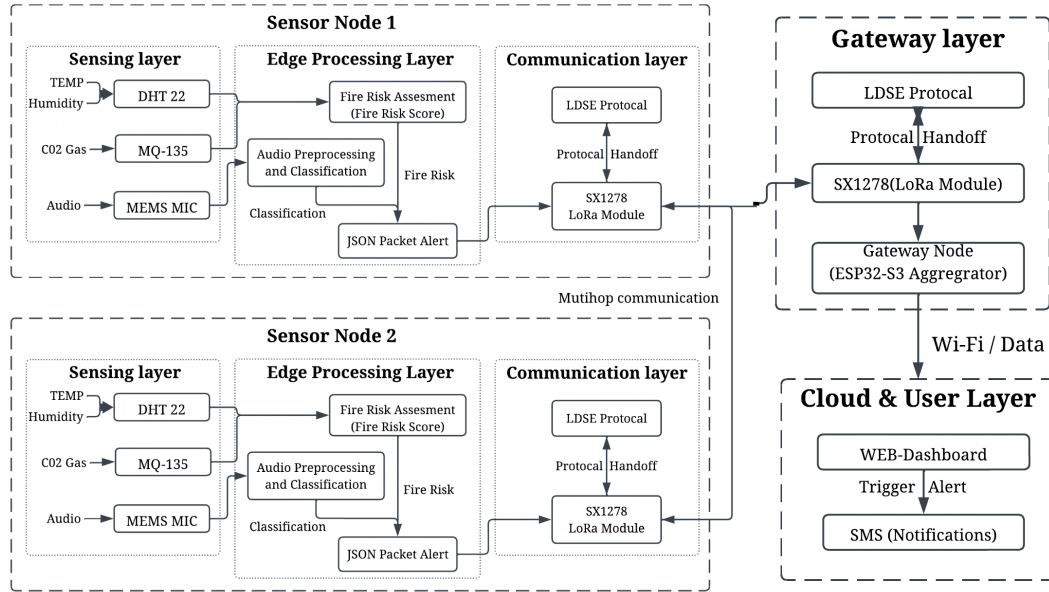


Figure 4-1: Proposed System Architecture

4.2 Communication System Design

To overcome the coverage limitations of traditional LoRaWAN (single-hop) and the high control overhead of reactive protocols like AODV, the network adopts the **Layered**

Dynamic Synchronization Energy-saving (LDSE) protocol.[6] In standard single-hop LoRaWAN, a node positioned several kilometres deep in a jungle must scale its Spreading Factor up to SF12 to overcome heavy canopy attenuation, spiking the Time-on-Air and draining the battery with every transmission. LDSE solves this by structuring the forest into logical, concentric tiers wrapped around one or more central gateway proxies, so each node communicates only with its nearest parent tier using short, low-power hops at optimised spreading factors such as SF7 or SF8.

4.2.1 LoRa Communication Fundamentals

LoRa (Long Range) is a proprietary, physical-layer wireless technology developed by Semtech that enables long-range, low-power, and low-data-rate connectivity for battery-powered IoT devices. It is based on Chirp Spread Spectrum (CSS) modulation, a technique in which the carrier frequency is swept linearly over time to encode information. CSS provides high receiver sensitivity, strong resistance to interference, and reliable communication under low signal-to-noise ratio (SNR) conditions, making it well suited to large-scale environmental monitoring applications such as forest sensing, where nodes are widely dispersed and often obstructed by dense vegetation.

4.2.2 LoRa Communication Parameters

The communication performance of a LoRa link is governed primarily by four parameters, each involving a trade-off between range, data rate, latency, and energy consumption.

- **Spreading Factor (SF):** Determines how much a signal is “spread” in the frequency domain before transmission. A higher SF (e.g., SF12) increases communication range and link robustness at the cost of a longer packet Time-on-Air and higher energy consumption, whereas a lower SF (e.g., SF7) offers faster transmission and lower energy use at the cost of reduced range.
- **Bandwidth (BW):** The width of the frequency channel used for transmission (commonly 125 kHz, 250 kHz, or 500 kHz in the sub-GHz ISM bands). A wider

bandwidth increases data rate and reduces Time-on-Air but lowers receiver sensitivity, shortening effective range.

- **Coding Rate (CR):** The ratio of forward error correction (FEC) redundancy added to a packet. A higher coding rate improves resilience to burst interference and multipath fading at the expense of additional transmission overhead and airtime.
- **Transmission Power:** The radiated output power of the transceiver. Higher transmission power extends range but increases energy consumption per packet and, in ISM bands, is often constrained by regional duty-cycle and power regulations.

Selecting appropriate values for these parameters therefore represents an important design trade-off between coverage, reliability, latency, and battery life, and directly motivates the multi-hop, low-SF approach adopted by LDSE (Section 4.2.5).

4.2.3 Conventional LoRaWAN Architecture

LoRaWAN defines the networking protocol operating above the LoRa physical layer. A conventional LoRaWAN deployment follows a *Star-of-Stars* topology, in which multiple end devices communicate directly with one or more gateways. These gateways transparently forward received packets to a central Network Server, which performs device authentication, duplicate packet removal, adaptive data rate management, and routing toward the Application Server.

This architecture offers simple deployment, centralised management, and low maintenance overhead, making it well suited to applications such as smart agriculture, smart cities, and general-purpose environmental sensing. However, communication remains inherently single-hop: every sensor node must maintain a direct wireless link to a gateway, regardless of terrain or vegetation density.

4.2.4 Limitations of Conventional LoRaWAN

Although conventional LoRaWAN provides excellent communication range under open-terrain conditions, dense forests introduce severe propagation losses due to vegetation, tree trunks, canopy shadowing, and irregular topography. Maintaining a direct gateway link under these conditions typically requires raising the spreading factor, which significantly increases packet airtime, transmission energy, and channel occupancy. Furthermore, deploying additional gateways to compensate for coverage gaps substantially increases infrastructure cost — a burden that is particularly impractical for low-budget community forest deployments in Nepal.

These constraints define the coverage-versus-cost trade-off that motivates the shift away from a single-hop star topology toward an energy-efficient multi-hop architecture.

4.2.5 LDSE Protocol Overview

To overcome the limitations described above, this project adopts the Layered Dynamic Synchronization Energy-saving (LDSE) protocol. Rather than relying on long-distance single-hop communication, LDSE establishes an energy-efficient multi-hop hierarchy in which nearby nodes relay data through synchronised intermediate parents. This approach extends network coverage, improves link reliability under forest propagation conditions, reduces transmission energy, and enables scalable deployment over large monitoring areas while maintaining low-power operation.

LDSE organises communication through three clear structural phases: **Layer Assignment**, **Dynamic Time-Synchronisation**, and **Time-Slotted Contention-Free Data Transmission**, detailed in Sections 4.2.6–4.2.9.

4.2.6 Architecture of LDSE

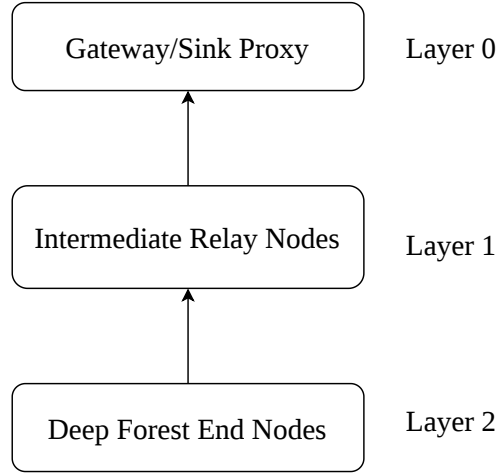


Figure 4-2: LDSE protocol network hierarchy

Layered Topology: The gateway (Layer 0) broadcasts an initialisation beacon during network setup. Each node assigns itself a hierarchical layer by incrementing the layer value carried in the beacon it receives:

$$L_{\text{node}} = L_{\text{received}} + 1 \quad (4.1)$$

This cascading propagation forms a tree-structured topology across the entire deployment area without any centralised configuration, as shown in Figure 4-2.

When a forest node is more than 2 km from the gateway, alert packets are automatically forwarded through intermediate relay nodes operating under LDSE routing, extending effective network coverage without the need for additional gateways.

4.2.7 Time Synchronization and Epoch Structure

Time-Slotted Epoch Structure: LDSE divides time into global operational epochs, each subdivided into three non-overlapping windows: a *Synchronisation Window* in which all transceivers wake simultaneously to receive time-alignment correction beacons from parent layers; a *Data Transmission Window* implementing TDMA-gated up-links layer by layer; and a *Deep-Sleep Period* during which all microcontrollers and radio transceivers enter ultra-low-leakage power states consuming as little as $7\mu\text{A}$.

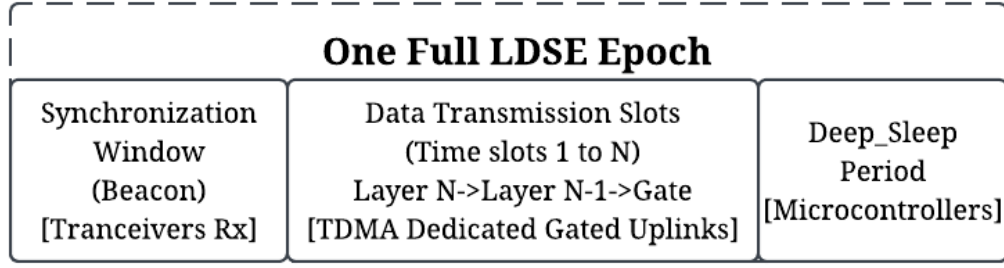


Figure 4-3: LDSE Protocol Epoch Structure

Time Synchronisation: The Flooding Time Synchronisation Protocol (FTSP) distributes a global timestamp from the gateway to all nodes, correcting clock drift caused by ambient temperature variation in the forest canopy and ensuring nodes do not miss their relay windows while remaining in deep sleep for the majority of each cycle.

4.2.8 Parent Selection and Routing Mechanism

Parent Selection and Implicit Route Exploration (IRE): When a node hears multiple candidate parents within a higher tier, it selects the optimal upstream relay using a combined score that balances link quality against remaining relay energy:

$$\text{Score} = \alpha \cdot \text{RSSI}_{\text{link}} + \beta \cdot \left(1 - \frac{E_{\text{consumed}}}{E_{\text{initial}}} \right) \quad (4.2)$$

Routes are discovered through active beaconing or passive eavesdropping on existing transmissions, limiting route-discovery overhead to approximately 5% of total traffic and avoiding the broadcast storms characteristic of AODV.

4.2.9 Multi-Channel Communication

Multi-Channel Separation: A public channel carries handshake and synchronisation messages, while a private channel carries payload data, avoiding intra-network collisions and congestion at relay nodes.

4.2.10 Power Consumption Model

The energy savings of LDSE arise from exchanging extended receiver listening times for drastically reduced transmission times. The total energy consumed by a node during one active operational cycle is:

$$E_{\text{total}} = E_{\text{sleep}} + E_{\text{sense}} + E_{\text{processing}} + E_{\text{sync}} + E_{\text{tx}} + E_{\text{rx}} \quad (4.3)$$

The dominant saving comes from the transmission term. Comparing a standard single-hop link at SF12 against an LDSE multi-hop link at SF7 over a 3 km forested path:

$$T_{\text{toA}_{\text{SF12}}} \approx 983 \text{ ms}, \quad T_{\text{toA}_{\text{SF7}}} \approx 56 \text{ ms}$$

$$E_{\text{tx, single}} = 3.3 \text{ V} \times 120 \text{ mA} \times 0.983 \text{ s} \approx 389 \text{ mJ}$$

$$E_{\text{tx, LDSE}} = 3.3 \text{ V} \times 45 \text{ mA} \times 0.056 \text{ s} \approx 8.3 \text{ mJ}$$

This represents a transmission energy reduction of approximately $46.8\times$. Even after accounting for the receiver overhead incurred by relay nodes ($\approx 1.65 \text{ mJ}$ per monitored child slot), the total network energy savings reach up to **42%** compared to classic single-hop topologies, allowing field-deployed nodes to operate reliably for several years on standard lithium iron phosphate (LiFePO_4) cells.

4.2.11 Forest Radio Propagation Considerations

Wireless communication in forest environments is strongly influenced by vegetation density, foliage absorption, terrain irregularities, and multipath propagation. Unlike free-space environments, dense forests introduce additional attenuation that increases with communication distance and canopy density. Consequently, signal strength degrades more rapidly, reducing communication reliability for long single-hop links.

During network simulation in ns-3, appropriate propagation models will be used to represent these environmental effects. The adoption of LDSE mitigates such attenuation by dividing long communication paths into multiple shorter relay links, thereby improving

packet delivery reliability while reducing the transmission power and airtime required by individual sensor nodes.

4.2.12 Protocol-Centric Implementation

This section outlines how to build and implement a functional two-tier LDSE protocol stack using an ESP32-S3 host microcontroller paired with an SX1262 LoRa transceiver module.

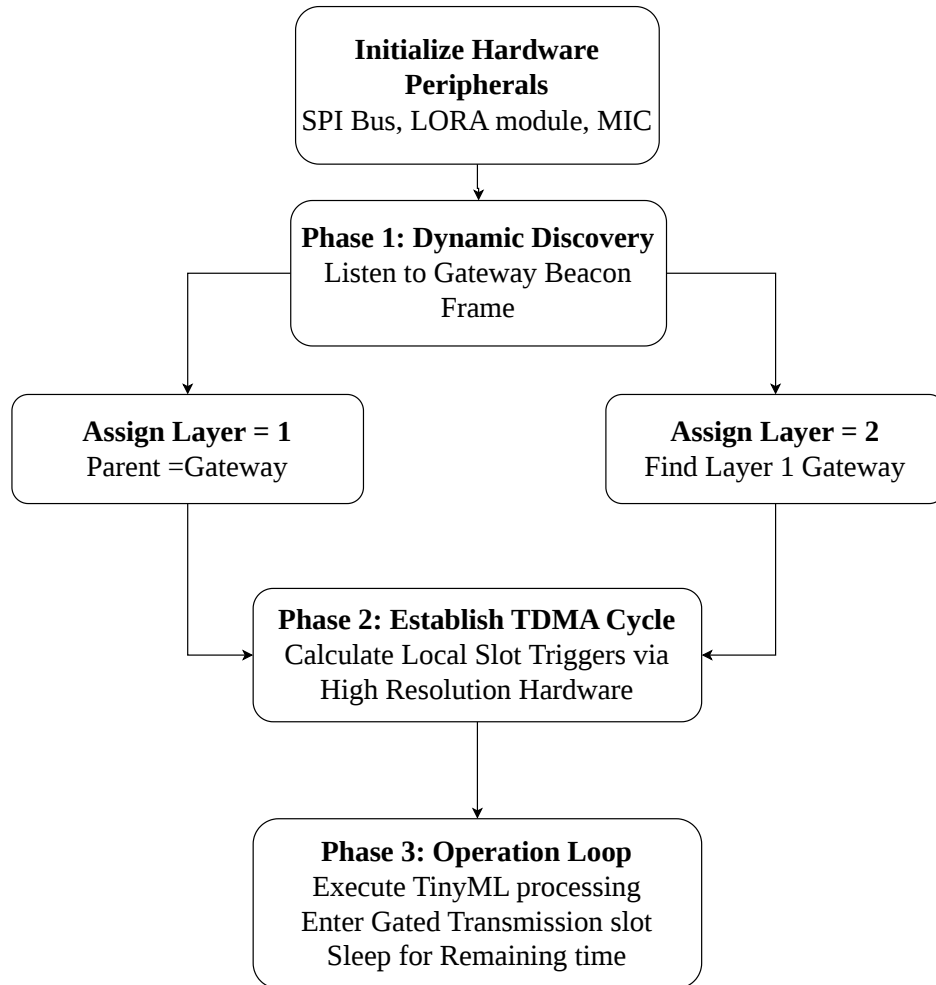


Figure 4-4: Sequential Synchronization and Operational Flowchart

4.2.13 Operational Challenges and Mitigation Strategies

Deploying LDSE in a dense forest environment introduces several engineering challenges, each addressed by a targeted mitigation:

- **Clock Drift due to Thermal Variation:** Diurnal temperature swings cause low-cost quartz oscillators to drift by several milliseconds over a few hours, risking overlapping TDMA slots and packet collisions. This is addressed by inserting a Guard Time Interval ($\tau_g = 20\text{--}50\text{ms}$) at the edges of each slot and by using Temperature-Compensated Crystal Oscillators (TCXO) on custom hardware boards to maintain timing alignment across temperature shifts.
- **Relay Congestion during Mass-Alert Events:** During a fast-spreading wildfire, many outer nodes may send alerts simultaneously, saturating the memory buffers of Layer 1 relay nodes. An asynchronous priority queue combined with a dynamic spreading-factor scale-up mechanism addresses this: when a relay's buffer exceeds 80% capacity, it sets a congestion flag in its synchronisation beacon, signalling child nodes to switch to a higher spreading factor (e.g., SF10) and route alerts directly to the gateway, temporarily bypassing the busy relay tier.
- **Hidden Terminal Collisions:** Two sibling nodes in the same layer may be out of range of each other yet share the same parent relay. If both transmit simultaneously, their signals collide at the relay, corrupting both packets. A randomised exponential back-off delay within the assigned TDMA slot window, combined with LoRa Channel Activity Detection (CAD), which mitigates this: before transmitting, each node listens for two symbol durations to detect any active preamble chirp on the channel and defers if the channel is busy.

4.2.14 Acoustic Dataset: FSC22

The acoustic threat classifier is trained on a curated subset of the FSC22 (Forest Sound Classification 2022) dataset [13]. FSC22 contains 2,025 five-second audio clips across 27 sub-classes grouped under six broader categories, each holding 75 samples drawn from the Freesound library.

For this project, FSC22 was narrowed to six project-relevant classes: Axe, Chainsaw, Gunshot, Handsaw, Tree Falling, and a merged Background class combining twelve non-threat sub-classes (rain, wind, stream, birds, insects, footsteps, and others). Ta-

ble 4-1 shows this mapping. All non-forest classes (Car, Train, Engine, Motorcycle, Helicopter, etc.) were excluded as out of scope.

Table 4-1: FSC22 sub-class to project-class mapping

Project Class	Mapped FSC22 Sub-classes
Axe	Axe, Wood_Chop
Chainsaw	Chainsaw, Sawing, Wood_Sawing
Gunshot	Gunshot, Gunshot_Other
Handsaw	Handsaw, Sawing, Wood_Sawing
Tree Falling	Tree_Falling, Wood_Crack, Wood_Creak
Background	Rain, Wind, Stream, Ocean, Crackling_Fire, Steps, Footwalk, Bird, Bird_Other, Insects, Thunder, Church_Bell

This 6-class formulation was used for the Full and Compact architecture designs described below (Section 4.2.16). During training and evaluation of the deployed Tiny architecture, the Tree Falling class was dropped: its low-frequency crack/creak signature overlapped heavily with background forest noise and its removal improved the reliability of the remaining classes. The final Tiny model is therefore trained and evaluated on five classes – Axe, Chainsaw, Gunshot, Handsaw, and Background – and all classification results reported in Section 5.1.1 and Section 6.5 reflect this 5-class configuration.

4.2.15 Acoustic Feature Extraction

Raw 16 kHz audio is converted to a log-Mel spectrogram via a Hann window, FFT, and Mel filterbank, followed by per-frame normalisation. This time-frequency representation captures the harmonic content of chainsaw noise, the sharp transients of gunshots, and the impact profile of axe strikes.

4.2.16 Memory-Constrained Model Architecture Design

Designing a CNN for on-device inference on the ESP32-S3 required an iterative architecture search, since the peak intermediate tensor size at each convolution layer must fit within the chip’s 512 KB SRAM alongside the audio pipeline, FreeRTOS, and networking stack. Three architecture variants were designed, summarised in Table 4-2:

Full architecture: 5.0 s audio, 128 Mel bands, filter widths [32,64,64]. The first convolution layer alone produces an intermediate tensor of approximately 2.05 MB – more than four times the total SRAM budget, making this architecture undeployable regardless of quantization.

Compact architecture: Audio duration halved to 2.5 s, Mel bands halved to 64, filters reduced to [16,32,32], dense layer reduced to 64 units. This reduces the peak intermediate tensor to approximately 257 KB – still impractical once the log-Mel buffer, FFT buffers, FreeRTOS, I2S DMA, and Wi-Fi stack overhead are accounted for.

Tiny architecture (selected for implementation): Filters reduced further to [8, 16, 32], and GlobalAveragePooling2D used in place of Flatten+Dense to avoid a large flattened feature vector. This reduces the peak intermediate tensor to approximately 128 KB, with a target tensor arena in the 90–212 KB range depending on safety margin.

Table 4-2: Model architecture design comparison

Parameter	Full	Compact	Tiny
Audio duration	5.0 s	2.5 s	2.5 s
Mel bands	128	64	64
Conv filters	[32, 64, 64]	[16, 32, 32]	[8, 16, 32]
Parameters (approx.)	65,000	24,000	9,000
Peak intermediate tensor	~2.05 MB	~257 KB	~128 KB

4.2.17 CNN Architecture

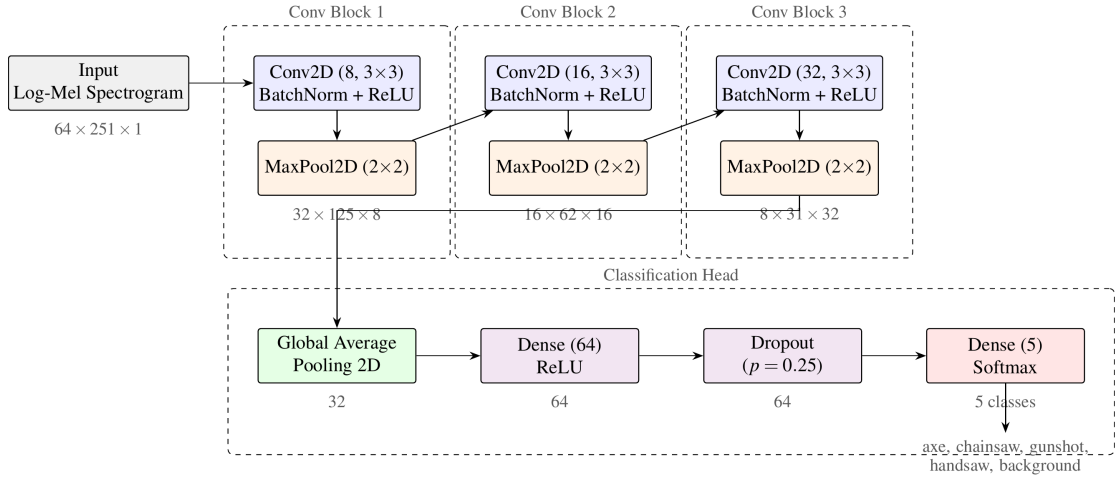
The deployed Tiny model is a compact convolutional neural network that maps a single $64 \times 251 \times 1$ log-Mel spectrogram to a probability distribution over the five project classes (axe, chainsaw, gunshot, handsaw, background). It consists of three convolutional blocks followed by a lightweight classification head, summarised in Table 4-3 and illustrated in Figure 4-5.

Each convolutional block applies a 3×3 Conv2D layer with same padding (so the spatial resolution is unchanged by the convolution itself), followed by BatchNormalization to stabilise activation statistics during training, a ReLU non-linearity, and a 2×2 MaxPooling2D layer that halves both spatial dimensions. The three blocks progressively widen the channel dimension ($8 \rightarrow 16 \rightarrow 32$ filters) while shrinking the frequency-time resolution from 64×251 down to 8×31 , following the standard CNN design pattern of trading spatial resolution for feature depth.

Rather than flattening the final $8 \times 31 \times 32$ feature map into a large fully-connected layer – which would require a $8 \times 31 \times 32 = 7,936$ -element weight vector per output neuron – the network applies GlobalAveragePooling2D, collapsing each of the 32 channels to a single average value. This keeps the head of the network small (a 32-element vector) regardless of the spectrogram’s time-frequency resolution, which is what makes the Tiny variant deployable within the ESP32-S3’s memory budget (Section 4.2.16). A Dense(64) layer with ReLU activation and Dropout(0.25) forms the hidden classification layer, and a final Dense(5) layer with softmax activation produces the class probabilities. In total the network has approximately 8,500 trainable parameters (rounded to $\sim 9,000$ in Table 4-2), the great majority of which are concentrated in the third convolutional layer and the first dense layer.

Table 4-3: Layer-by-layer summary of the deployed Tiny CNN

Layer	Type	Output Shape	Parameters
Input	Log-Mel spectrogram	$64 \times 251 \times 1$	–
Conv Block 1	Conv2D(8, 3×3) + BN + ReLU	$64 \times 251 \times 8$	112
	MaxPool2D(2×2)	$32 \times 125 \times 8$	0
Conv Block 2	Conv2D(16, 3×3) + BN + ReLU	$32 \times 125 \times 16$	1,232
	MaxPool2D(2×2)	$16 \times 62 \times 16$	0
Conv Block 3	Conv2D(32, 3×3) + BN + ReLU	$16 \times 62 \times 32$	4,768
	MaxPool2D(2×2)	$8 \times 31 \times 32$	0
Head	GlobalAveragePooling2D	32	0
	Dense(64) + ReLU	64	2,112
	Dropout($p = 0.25$)	64	0
	Dense(5) + Softmax	5	325
Total			$\sim 8,549$

**Figure 4-5:** Block diagram of the deployed Tiny CNN, showing the three convolutional blocks and the classification head

4.2.18 Quantization Strategy

Post-training INT8 quantization is applied to each candidate model using TensorFlow Lite's converter, with a representative calibration dataset used to fix activation ranges

ahead of deployment on target hardware. INT8 quantization is required because the ESP32-S3 has no hardware floating-point acceleration suited to real-time CNN inference, and materially reduces both model size and inference latency compared to a float32 model.

4.2.19 On-Device Deployment Pipeline

Figure 4-6 shows the firmware-level inference pipeline running on each forest node. The I2S peripheral captures 40,000 samples (2.5 s at 16 kHz) from an INMP441 MEMS microphone into a heap-allocated `int16_t` ring buffer. This buffer is converted into a 64×251 log-Mel spectrogram (Hann window, 512-point FFT, 64-band Mel filterbank, log compression, per-frame normalisation), quantized to INT8 using the input scale and zero-point and passed to the TFLite Micro interpreter's `Invoke()` call. The raw INT8 output logits are dequantized back to float probabilities ($\text{prob} = (\text{output}_i - \text{zp}) \times \text{scale}$), and the argmax class is taken as the detection result.

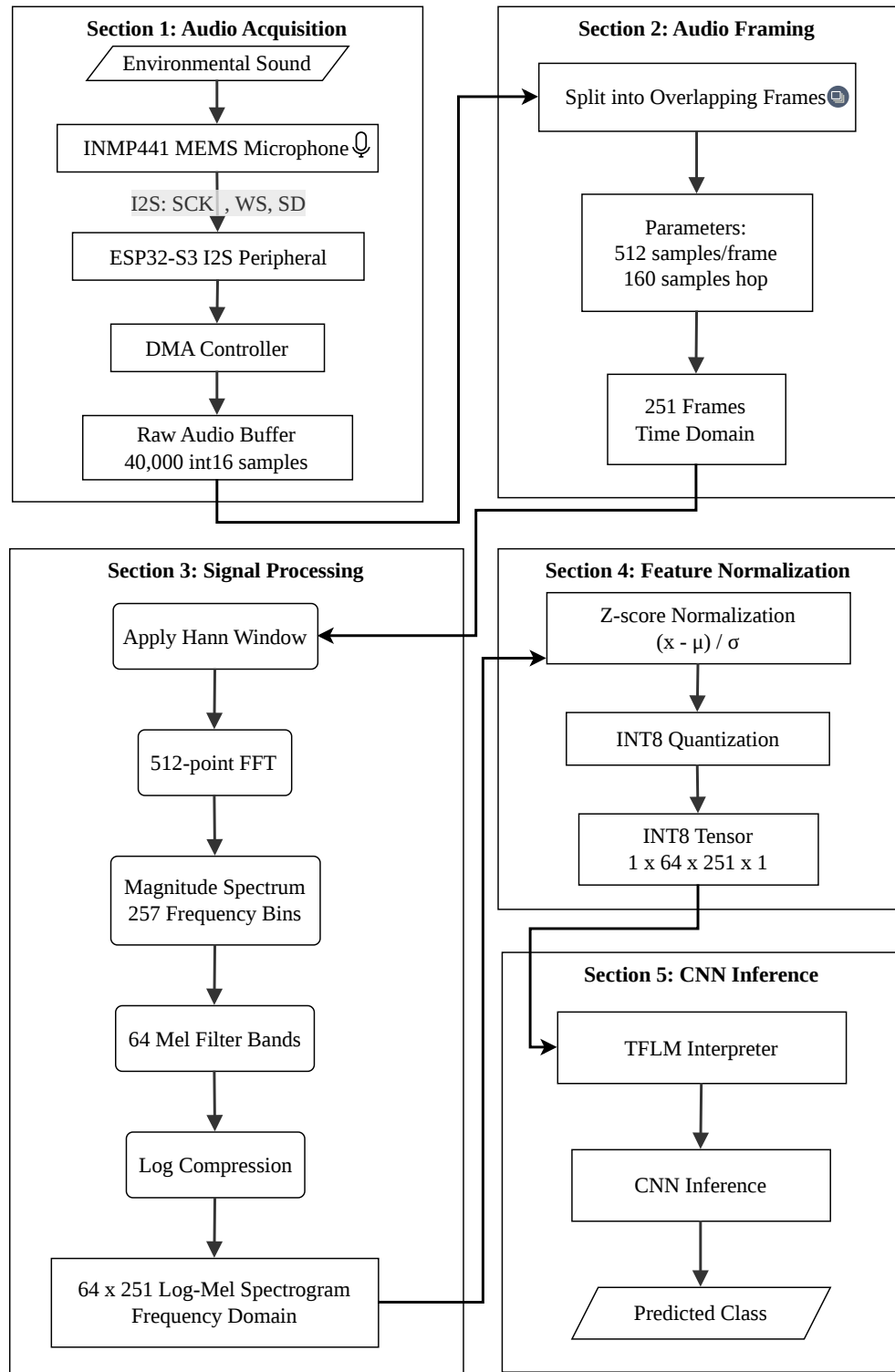


Figure 4-6: On-device acoustic inference pipeline

4.2.20 Fire Risk Scoring

Instead of simple fixed threshold comparisons, each node computes a composite fire-risk score by multisensory fusion, measuring how much current sensor readings deviate from normal (calibrated) conditions. The principle is:

- Normal forest conditions = stable temperature, CO₂, and humidity.
- Fire conditions = temperature spikes, CO₂ increases dramatically, humidity drops.

The system calculates a normalized deviation score, and if it exceeds a threshold, it signals a fire.

$$\text{FireRisk} = 0.51 \times \Delta\text{CO}_2 + 0.37 \times \Delta\text{TEMP} + 0.12 \times \Delta\text{HUMIDITY} \quad (4.4)$$

What each part means:

- $\Delta\text{CO}_2 = (\text{current CO}_2 - \text{Calibrated CO}_2) / \text{standard deviation of CO}_2$
- $\Delta T = (\text{current temp} - \text{Calibrated temp}) / \text{standard deviation of temp}$
- $\Delta H = (\text{current humidity} - \text{Calibrated humidity}) / \text{standard deviation of humidity}$

Weights (based on correlation with fire):

- 0.51 (51%) = CO₂ (highest correlation: 0.4842)
- 0.37 (37%) = Temperature (correlation: 0.3477)
- 0.12 (12%) = Humidity (lowest correlation: 0.10)

Fire likely detected: risk score is above 3.0 threshold.

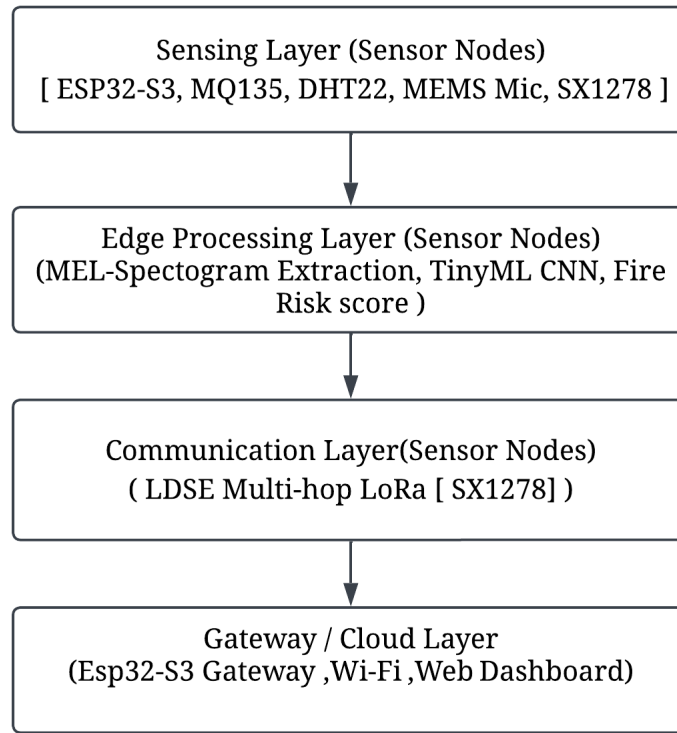


Figure 4-7: Proposed System Dataflow

4.3 System Data Flow

1. **Sensing Layer:** Forest nodes maintain deep sleep (approximately $7 \mu\text{A}$ draw). Nodes wake on a scheduled 10-minute timer for environmental sampling, or immediately upon a MEMS microphone An interrupt triggered by audio energy above the silence threshold.
2. **Edge Processing:** The ESP32-S3 wakes, reads MQ135 and DHT22 sensors, and compute the RF fire-risk score. Simultaneously or Upon a microphone interrupt, it captures a 5-second audio window, extracts The Mel-spectrogram runs the quantized CNN inference. If either the fire-risk score exceeds the safety threshold or the CNN inference confidence exceeds the acoustic detection threshold (default 0.85), the node forms a JSON alert packet containing node ID, GPS coordinates, timestamp, risk score, and detected class label.
3. **Communication Layer:** The JSON alert packet is transmitted over LoRa. If the node is within direct range of the gateway ($< 2 \text{ km}$), The packet is sent directly. If

beyond range, the LDSE routing layer forwards the packet through intermediate relay nodes in a multi-hop chain until it reaches the gateway.

4. **Gateway / Cloud Layer:** The ESP32-S3 gateway receives the alert packet, performs optional secondary validation (duplicate filtering, minimum confidence check), and forwards the structured alert to the web dashboard over Wi-Fi for visualization and notification of forest authorities.

4.4 Overall Working Principle

The network operates in a closed-loop, event-driven manner. Scheduled environmental sampling provides background fire-risk monitoring, while interrupt-driven acoustic wake-up enables immediate response to chainsaw, gunshot, or axe sounds. Edge inference before transmission ensures that only confirmed threat events generate network traffic, significantly reducing energy consumption compared to systems that stream raw sensor data continuously. LDSE synchronized sleep–relay cycles ensure that relay nodes are awake to forward packets precisely when needed, while remaining in deep sleep otherwise. The gateway publishes structured alerts to the web dashboard in real time, enabling forest rangers to identify the location and nature of each detected event and respond accordingly.

5 IMPLEMENTATION DETAILS

5.1 Acoustic Threat Detection Implementation

5.1.1 Training Pipeline

The acoustic model development pipeline was implemented independently in Python using TensorFlow/Keras, rather than the Edge Impulse platform originally proposed. This gave full control over preprocessing, the architecture search described in Section 4.2.16, and quantization – control that proved necessary once memory profiling on target hardware revealed how extensively the model had to be redesigned.

Data Preparation: The 5-class audio subset was split using a stratified 80/20 train-test split per class (`split_dataset.py`), computed on the original, unaugmented files to keep the test set clean. Augmentation (`augment_training.py`) – pitch shifting, time stretching, and additive noise injection – was applied only to the four threat classes (axe, chainsaw, gunshot, handsaw) and deliberately withheld from the background class, so that augmentation does not make background clips sound more threat-like. This expanded the augmented training set to roughly 360 files per threat class and 544 files for background.

Feature Extraction: Raw 16 kHz audio was converted to a log-Mel spectrogram using a Hann window, 512-point FFT, and a Mel filterbank, followed by per-frame normalisation (mean subtraction, unit standard deviation). Figure 5-1 shows example spectrograms for representative classes, illustrating the harmonic bands of chainsaw noise, the sharp broadband transient of a gunshot, and the impact profile of an axe strike.

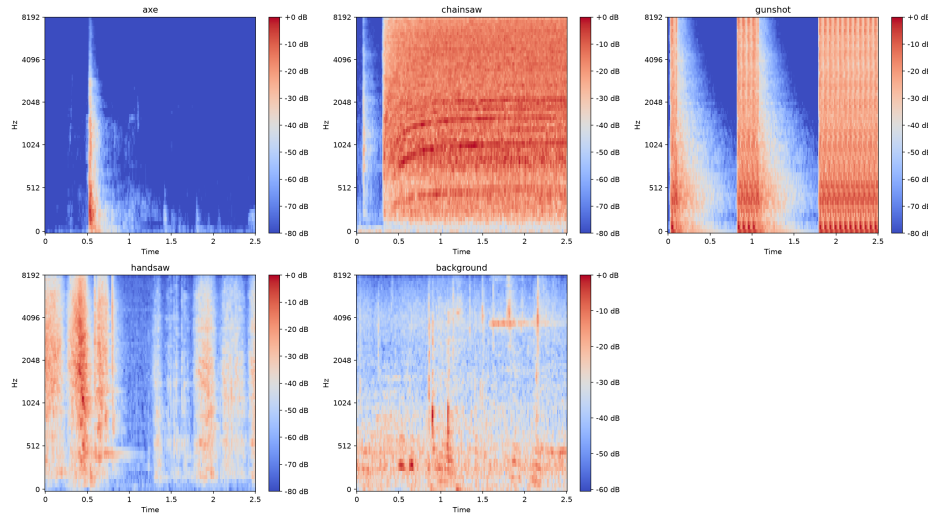


Figure 5-1: Log-Mel spectrograms for representative classes

Model Training: All three architecture variants (Section 4.2.16) were trained using categorical cross-entropy loss with the Adam optimizer. The final deployed (tiny) variant uses GlobalAveragePooling2D in place of a flattened dense layer, avoiding an oversized fully-connected weight matrix; its full layer-by-layer architecture and block diagram are detailed in Section 4.2.17.

5.1.2 INT8 Quantization

Each trained model was quantized to 8-bit integers using TensorFlow Lite’s post-training quantization API, with TFLITE_BUILTINS_INT8 as the target op set and INT8 fixed as both the inference input and output type:

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_ops = \
    [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
converter.representative_dataset = representative_dataset
```

A representative dataset of 100 randomly sampled training clips was used to calibrate activation quantization ranges. For the final deployed tiny model (retrained on the 5-class dataset), calibration produced an input scale of 0.076175 with zero-point -13 (mapping the INT8 range $[-128, 127]$ to approximately $[-8.8, 10.7]$ in the normalised float domain), and an output scale of 0.003906 with zero-point -128 for dequantizing softmax probabilities. This reduced the tiny model to a 19.24 KB INT8 TFLite file (`model_int8.tflite`), compiled into a C header (`model_data.h`) and flashed to the ESP32-S3 firmware. The resulting per-model file sizes and architecture parameters are summarised in Table 5-1.

Table 5-1: Deployed model size and parameter count by architecture variant

Metric	Full	Compact	Tiny (deployed)
Parameters	~65,000	~24,000	~9,000
TFLite (INT8) size	78 KB	27 KB	19 KB
Tensor arena	(undeployable)	(undeployable)	90 KB

Table 5-2 breaks down the full runtime memory budget on the ESP32-S3. The 90 KB tensor arena and 19 KB of flash-resident model weights leave sufficient headroom for the remaining audio pipeline, FreeRTOS, and networking stack within the 512 KB SRAM budget.

Table 5-2: ESP32-S3 runtime memory budget

Component	Size	Region
Model weights	19 KB	Flash
Tensor arena	90 KB	DRAM
Log-Mel buffer (int16)	32 KB	Heap
FFT + magnitude	5 KB	Heap
I2S DMA	8 KB	Internal
Ring buffer + stack	~2 KB	Internal

5.2 LoRa Network Simulation Implementation

To evaluate the wireless communication capability of the proposed forest monitoring system, a LoRaWAN-based simulation environment was developed using OMNeT++ along with the FLORA framework and the INET library.

The simulation environment consists of:

- LoRa end devices.
- LoRa gateway.
- LoRa communication medium.
- Network communication components provided by the INET framework.

A basic LoRa communication scenario was implemented where LoRa sensor nodes communicate through a LoRa gateway.

The simulation workflow includes:

1. Creation and configuration of LoRa nodes using the FLORA framework.
2. Configuration of LoRa physical layer parameters.
3. Packet generation from the end devices.
4. Wireless packet transmission through the LoRa medium.
5. Reception and processing of packets at the gateway.

The current simulation successfully validates basic LoRa communication between sensor nodes and provides a foundation for implementing the proposed LDSE multi-hop routing protocol in future work.

The implemented LoRa network topology in OMNeT++ is illustrated in Fig. 5-2.

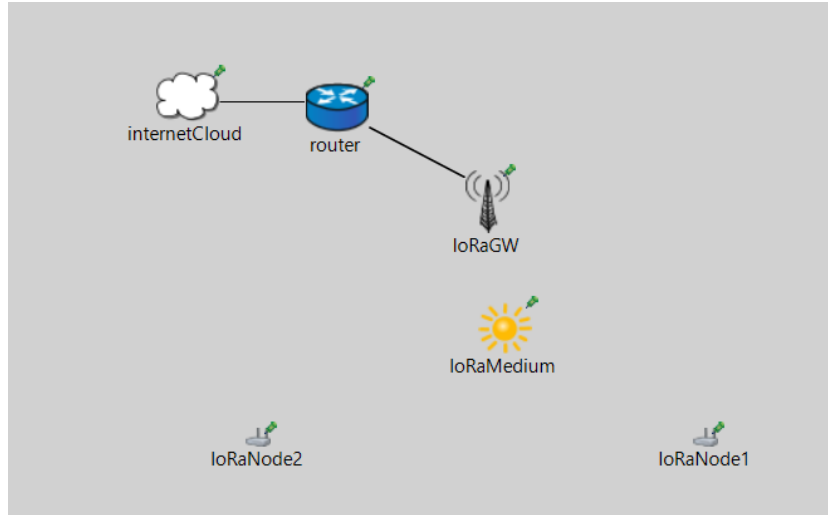


Figure 5-2: LoRaWAN communication using OMNeT++

5.2.1 Basic LoRa Node Communication Implementation

Before implementing the proposed multi-hop routing mechanism, a basic LoRa communication scenario was developed to verify the operation of individual LoRa end devices. In this implementation, two LoRa sensor nodes were configured to exchange data through the LoRa wireless communication medium.

The first node acts as a LoRa transmitter, generating application data packets at predefined time intervals. These packets are processed through the LoRa protocol stack and transmitted using the LoRa physical layer. The second node acts as a LoRa receiver, which receives the transmitted packets and processes the incoming data.

The communication process consists of the following steps:

1. The LoRa transmitter node generates sensing data packets.
2. The packets are passed through the application and LoRa protocol layers.
3. The LoRa radio module performs wireless transmission using configured parameters such as spreading factor, bandwidth, and transmission power.
4. The receiving LoRa node detects and receives the transmitted packet.
5. The received data is delivered to the application layer for further processing.

This basic communication model validates the successful exchange of information between LoRa devices and confirms the proper configuration of the FLORA framework and INET communication modules. The implemented two-node LoRa communication scenario serves as the initial building block for extending the network towards gateway-based and multi-hop communication.

The implemented two-node LoRa communication topology is shown in Fig. 5-3.

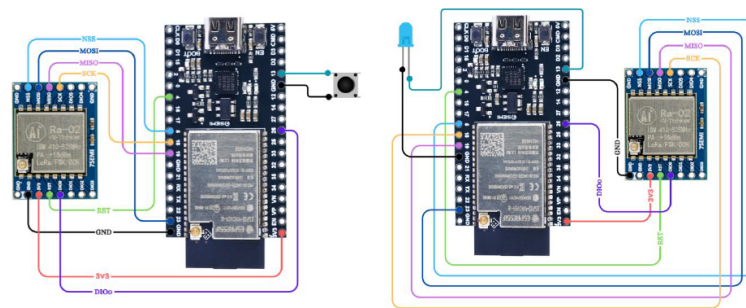


Figure 5-3: LoRa communication between two LoRa end devices in OMNeT++

6 RESULT AND ANALYSIS

6.1 Environmental Sensor-Based Fire Risk Assessment

The developed sensor prototype was tested under different environmental conditions to verify the fire risk estimation mechanism.

The sensor node successfully collected temperature, humidity, and gas concentration data and generated a corresponding fire risk score.

Example observations obtained from testing are shown in Table 6-1.

Table 6-1: Environmental Sensor Testing Results

Condition	Temperature	Humidity	Gas Level	Risk Level
Normal Environment	Low	High	Low	Low
Warm and Dry	Medium	Medium	Medium	Moderate
Smoke Exposure	High	Low	High	High

The results indicate that combining multiple sensor parameters provides a more reliable indication of fire risk compared to individual sensor threshold methods. The sensor fusion approach reduces false alarms caused by temporary variations in a single environmental parameter.

6.2 LoRa Communication Simulation Results

The LoRa communication simulation successfully demonstrated packet transmission between LoRa sensor nodes using the FLORA framework.

The implemented simulation verified:

- Successful packet transmission between LoRa nodes.
- Reception of packets at the LoRa gateway.
- Basic wireless communication functionality.

The simulation confirms that the FLORA and INET-based environment can be used for evaluating the proposed forest monitoring network.

The future simulation work will include:

- Implementation of LDSE multi-hop routing protocol.
- Energy consumption analysis.
- Packet delivery ratio evaluation.
- Network scalability testing.
- Performance comparison with existing routing approaches.

6.3 Current Limitations

Although the initial prototype and simulation environment have been successfully developed, some components of the complete system are still under development.

The remaining tasks include:

- Deployment of TinyML-based acoustic classification model.
- Implementation of LDSE routing protocol.
- Hardware-based LoRa communication testing.
- Cloud dashboard integration.
- Large-scale network simulation.

6.4 Current Project Progress

Table 6-2: Current Project Status

Module	Status
ESP32-S3 Sensor Node	Completed
Temperature/Humidity/Gas Sensing	Completed
Fire Risk Score Calculation	Completed
Acoustic Data Acquisition	Completed
TinyML Sound Classification	In Progress
LoRa Hardware Communication	In Progress
FLORA LoRa Simulation	Completed
LDSE Routing Protocol	Future Work
Web Dashboard	Future Work

6.5 Acoustic Threat Detection Results

The deployed `model_int8.tflite` model was evaluated on the held-out FSC22 test split (225 samples, stratified across the 5 project classes described in Section 4.2.16). The model achieved an overall accuracy of **88.89%** (200/225 correct), a balanced accuracy of **82.42%**, and a micro-averaged AUC-ROC of **0.9838**, confirming that the Tiny architecture – the only variant small enough to fit the ESP32-S3’s memory budget (Table 4-2) – remains highly discriminative despite its reduced parameter count.

Table 6-3 reports precision, recall, and F1-score per class. Background – the majority class with 165 test samples – is classified with very high precision and recall (0.9620 / 0.9212), which is the desired behaviour for a safety-critical detector, since false positives on background would generate nuisance alerts.

Table 6-3: Classification report – Tiny INT8 model on FSC22 test split

Class	Precision	Recall	F1-score	Support
axe	0.5789	0.7333	0.6471	15
chainsaw	0.8571	0.8000	0.8276	15
gunshot	0.6842	0.8667	0.7647	15
handsaw	0.8000	0.8000	0.8000	15
background	0.9620	0.9212	0.9412	165
Accuracy		0.8889		225
Macro avg	0.7765	0.8242	0.7961	225
Weighted avg	0.9002	0.8889	0.8928	225

Figure 6-1 visualises the same precision, recall, and F1-score values, making the weaker classes easier to spot at a glance: axe has the lowest precision (0.5789 vs. 0.7333 recall), and gunshot shows a similar precision gap (0.6842 precision vs. 0.8667 recall) – suggesting the model is currently biased toward flagging ambiguous impulsive sounds as axe or gunshot strikes rather than missing them, an acceptable trade-off for a security application.

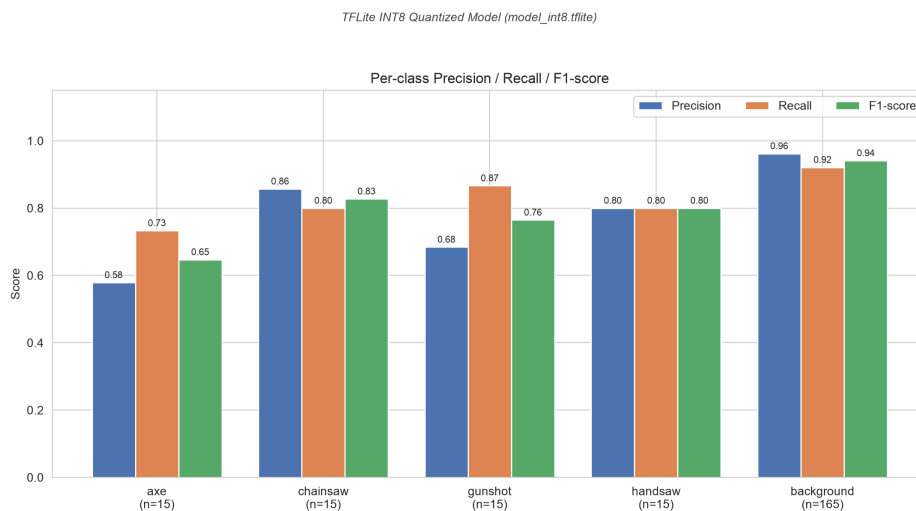
**Figure 6-1:** Per-class precision, recall, and F1-score for the deployed tiny model

Figure 6-2 shows the confusion matrix underlying these scores. Axe's precision gap is

driven mainly by confusion with background: three of fifteen axe clips were misclassified as background, plausibly because the sharp broadband impact of an axe strike is brief and can be masked by ambient forest noise once averaged over the 2.5 s window; conversely, five background clips were misclassified as axe, and four as gunshot, reflecting the acoustic overlap between short, impulsive background events (e.g. falling debris, distant cracks) and the transient signatures of the threat classes.



Figure 6-2: Confusion matrix for the deployed tiny INT8 model)

Despite this per-class variation, the model separates every class from the rest well in ranking terms, achieving a micro-averaged AUC-ROC of 0.9838 and a per-class one-vs-rest AUC no lower than 0.9370 (gunshot) – indicating that the confidence threshold, rather than the model’s underlying discriminative power, is the primary lever for improving the weaker classes before final deployment.

7 FUTURE ENHANCEMENTS

Based on the current project status, the following tasks remain to complete the minor project. The work is divided into two parts: Part A covers the forest fire and acoustic threat detection hardware and TinyML pipeline, and Part B covers the LoRa communication and LDSE protocol simulation.

Part A: Forest Fire & Acoustic Threat Detection (Hardware & TinyML)

This portion of the project focuses on collecting environmental data, processing audio on the edge, and calculating localized hazard levels.

Edge Hardware and Sensor Integration

- **Circuit Assembly:** Wire the core sensors (DHT22 for temperature/humidity and MQ135 for gas/smoke) and the MEMS microphone directly to the ESP32-S3 microcontroller.
- **Deep Sleep Optimization:** Program the ESP32-S3 to operate in deep sleep mode to conserve battery. It should wake up at fixed intervals (e.g., every 5 minutes) to sample environmental sensors, run the TinyML audio classifier, and broadcast results before returning to sleep.
- **Calibration:** Fine-tune MQ135 gas readings to ignore baseline environmental shifts and establish clean thresholds for smoke detection.

Acoustic Preprocessing and Model Deployment

- **Spectrogram Generation:** Implement local feature extraction in C++ on the ESP32-S3. Raw audio captured from the MEMS microphone must be converted into a Log-Mel spectrogram (using windowing, FFT, and a Mel-filterbank) matching the Python pipeline parameters.

- **Model Quantization and Loading:** Convert the trained TensorFlow model to a TensorFlow Lite Micro format. Apply INT8 quantization to compress the model size to fit the ESP32-S3's constrained SRAM and Flash memory.
- **On-Device Inference:** Run local inference on the ESP32-S3 to detect real-time threats (such as chainsaws, axe cuts, or gunshots) with low latency.

Fire Risk Algorithm and Sensor Fusion

- **Multisensor Fusion:** Write an on-device algorithm to calculate a dynamic “Fire Risk Score” using weighted inputs from the DHT22 and MQ135.
- **Threshold Triggering:** Set classification outputs to trigger immediate alert transmissions if the risk score crosses into the “High Risk” zone, bypassing standard sleep intervals.

Gateway and Dashboard Setup

- **Gateway Receiver Firmware:** Set up a receiving LoRa node that accepts incoming packets from the field and forwards them over Wi-Fi (via HTTP or MQTT).
- **Live Web Dashboard:** Build a central monitoring web page to display incoming sensor values (temperature, humidity, gas), live fire risk level, localized acoustic threat alerts, and device battery status.

Part B: LoRa Communication & Protocol Simulation (Core Part)

This portion transitions the current OMNeT++/FLORA simulation setup over to **ns-3** to implement and test the LDSE multi-hop routing protocol.

ns-3 Environment Setup and Channel Modeling

- **Simulation Migration:** Transition from OMNeT++ to an ns-3 environment, incorporating the ns-3 LoRaWAN module.

- **Forest Path Loss Model:** Implement a realistic wireless channel model that simulates forest conditions. This includes log-distance path loss combined with foliage attenuation factors to mimic how thick trees degrade LoRa signals compared to an open field.

LDSE Protocol Implementation

- **Dynamic Layer Formation:** Program nodes to automatically assign themselves to “layers” based on their distance (hop-count) from the central gateway.
- **Energy-Aware Parent Selection:** Implement the selection logic where a leaf node dynamically chooses its uplink relay parent based on path quality, link stability, and the parent node’s remaining battery.
- **Time Synchronization:** Set up dynamic beaconing to broadcast synchronization epochs, keeping the multi-hop network aligned in time to prevent slot collisions.
- **Slot-Based Scheduling and Sleep:** Program the sleep-wake schedules where relays and leaf nodes power down their radios outside of their assigned reception/transmission slots.

Performance Evaluation and Comparative Analysis

- **Scalability Testing:** Run simulations scaling from 20 up to 100 nodes across a virtual 1 to 5 km² forest grid.
- **Metrics Extraction:** Measure and export the following performance metrics:
 - **Packet Delivery Ratio (PDR):** Percentage of generated packets successfully reaching the gateway.
 - **End-to-End Latency:** Time taken for an alert to traverse the multi-hop network.
 - **Energy Consumption:** Average energy drain per node to assess total network lifetime.

- **Comparison:** Graph and analyze the results of the LDSE multi-hop protocol directly against standard single-hop LoRaWAN to prove its benefits in coverage extension and energy conservation.

8 CONCLUSION

This minor project investigated and demonstrated the feasibility of a scalable multi-hop LoRa network for intelligent forest monitoring, integrating environmental sensing, edge-based machine learning, and low-power communication into a single sensor node design.

The hardware prototype successfully collected temperature, humidity, and gas concentration data and combined them through multisensory fusion to compute a dynamic fire risk score. The testing results confirmed that combining multiple environmental parameters provides a more reliable indication of fire risk than individual sensor threshold methods and reduces false alarms caused by temporary fluctuations in a single reading.

The TinyML inference pipeline, evaluated on the training dataset, demonstrated that acoustic features could be effectively classified on-device, enabling the detection of threats such as chainsaws, axe cuts, and gunshots close to where the sound occurs. This edge-based approach avoids sending raw audio over the network, significantly reducing bandwidth and energy consumption.

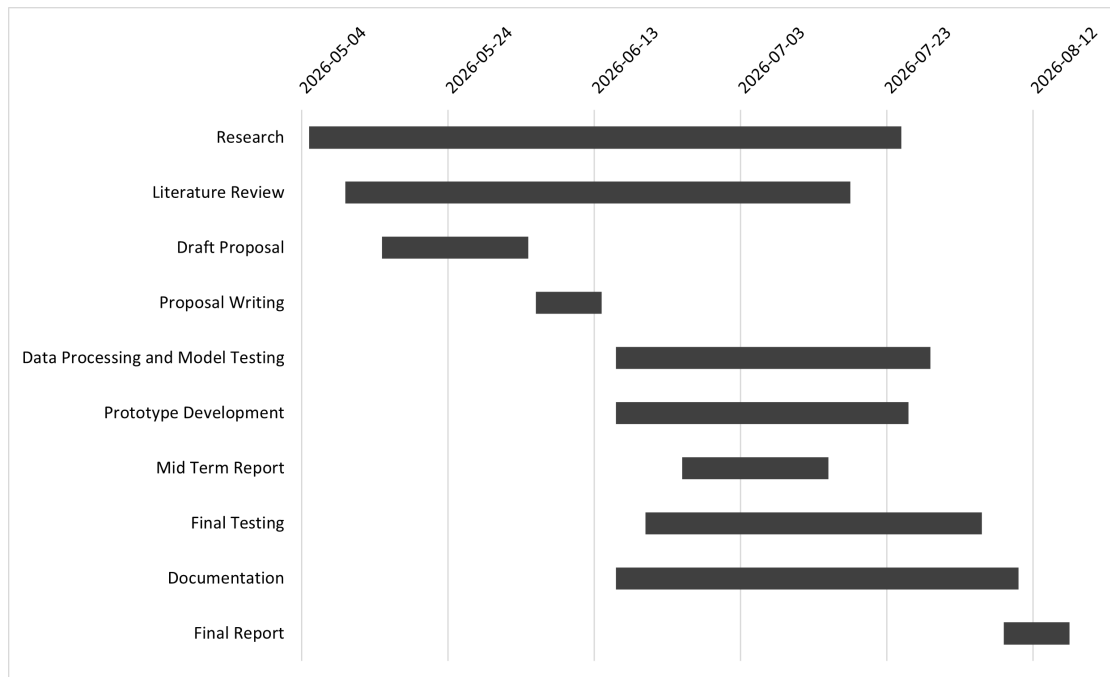
The LoRa communication simulation validated the ability of the network to extend coverage beyond the range of a single gateway. The proposed LDSE multi-hop protocol is expected to provide an energy-efficient, slot-scheduled routing scheme that maintains synchronization across the network while allowing nodes to remain in deep sleep outside their assigned time slots.

Overall, the design demonstrates that a low-cost, scalable, and energy-efficient solution based on LoRa and TinyML is achievable for real-time forest monitoring in community forests and protected areas. The remaining tasks described in the preceding chapter define the path toward a complete field-deployable implementation.

9 APPENDICES

Appendix A: PROJECT SCHEDULE

Table 9-1: Project Schedule Gantt Chart



Appendix B: PROJECT BUDGET

Table 9-2 presents the estimated budget for the hardware and development components required for the prototype implementation.

Table 9-2: Project Budget

S.N.	Component	Quantity	Unit Price (NPR)	Total (NPR)
1	ESP32-S3 Development Board	3	1600	4,800
2	LoRa Module (SX1278)	3	900	2,700
3	MQ135 Smoke/Gas Sensor	2	250	500
4	DHT22 Temperature-Humidity Sensor	2	400	800
5	MEMS Microphone Module (I2S)	2	600	1,200
6	Li-ion Battery	3	200	600
7	Enclosure and Mounting Hardware	3	300	900
8	Miscellaneous (PCB, wires, connectors, etc.)	3	300	900
Total Estimated Budget				12,400

Appendix C: HARDWARE PROTOTYPE SETUP

The hardware implementation details of the proposed forest monitoring sensor node are presented in this appendix. The prototype consists of an ESP32-S3 microcontroller, MEMS microphone, temperature and humidity sensor, gas sensor, and LoRa communication module.

The complete hardware connection diagram of the sensor node is shown in Fig. 9-1.

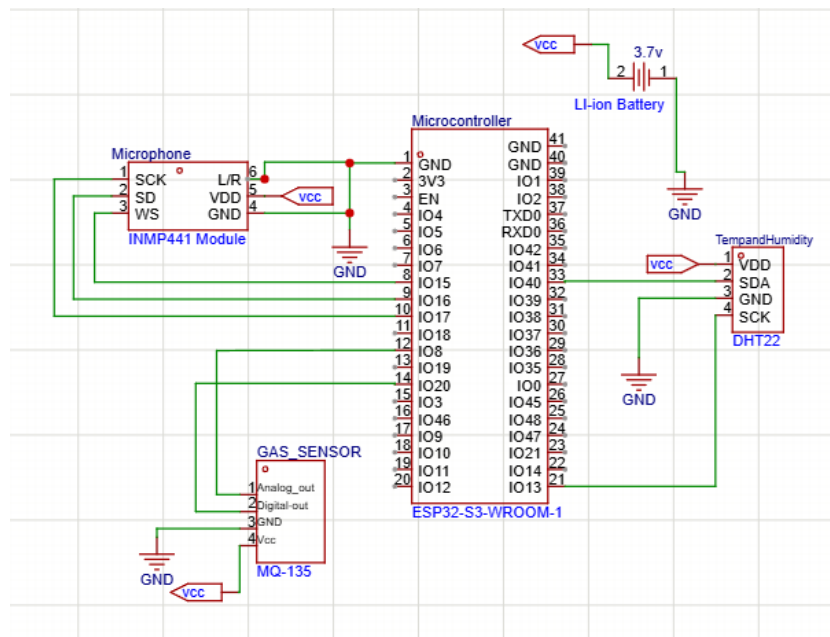


Figure 9-1: Schematic diagram of the sensing layer.

REFERENCES

- [1] M. A. Matin, V. S. Chitale, M. S. R. Murthy, K. Uddin, B. Bajracharya, and S. Pradhan, “Understanding forest fire patterns and risk in nepal using remote sensing, geographic information system and historical fire data,” vol. 26, no. 4, pp. 276–286.
- [2] B. Mishra, S. Panthi, S. Poudel, and B. Ghimire, “Forest fire pattern and vulnerability mapping using deep learning in nepal,” vol. 19, p. 3.
- [3] B. Bhattarai, K. Shrestha, P. Gurung, P. Sapkota, and H. Ojha, “Anticipatory forest-fire risk governance in nepal,” vol. 26, pp. 142–154.
- [4] P. W. Rundel, E. A. Graham, M. F. Allen, J. C. Fisher, and T. C. Harmon, “Environmental sensor networks in ecological research,” vol. 182, no. 3, pp. 589–607.
- [5] N. Javaid, S. Hayat, M. Shakir, M. A. Khan, S. H. Bouk, and Z. A. Khan, “Energy efficient MAC protocols in wireless body area sensor networks - a survey,”
- [6] Z. Li, X. Li, and J. Shang, “Forest fire monitoring and energy optimization based on LoRa-mesh wireless communication technology,” vol. 14, no. 21, p. 4135.
- [7] Semtech Corporation, “Semtech LoRa technology overview,”
- [8] N. Sornin, M. Luis, T. Eirich, T. Kramp, and O. Hersent, “LoRaWAN specification.” Version number: V1.0.
- [9] J. Haxhibeqiri, E. De Poorter, I. Moerman, and J. Hoebeke, “A survey of LoRaWAN for IoT: From technology to application,” vol. 18, no. 11, p. 3995.
- [10] M. Bandara, R. Jayasundara, I. Ariyaratne, D. Meedeniya, and C. Perera, “Forest sound classification dataset: FSC22,” vol. 23, no. 4, p. 2032.
- [11] A. K. Patel, P. Parikh, and P. Shah, “ForestGuard: An IP66 edge-AI raspberry pi node for illegal logging and early fire/smoke detection,” vol. 16, no. 2, pp. 1486–1500.

- [12] A. Saida, C. Sreedhar, Samreen, A. Mohamma, K. Jamal, and M. Ghalwan, "LORA based forest fire monitoring system," in *E3S web of conferences*, vol. 430, p. 01171, EDP Sciences.
- [13] S. Kahl, C. Wood, M. Eibl, and H. Klinck, "FSC22: A dataset for forest sound classification," vol. 23, no. 4, p. 2032.